Department of Computer Science

Lab Manual

of

DEEP LEARNING

MAI417-3

Class Name: III MScAIML

Master of Science Artifitial Intelligence and Machine Learning

2025-26

Prepared by: (your name) **Verified by: Faculty name:**

Department Overview

Department of Computer Science of CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape nation's destiny. The training imparted aims to prepare young minds for the challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field.

Vision

The Department of Computer Science endeavours to imbibe the vision of the University "**Excellence and Service"**. The department is committed to this philosophy which pervades every aspect and functioning of the department.

Mission

"To develop IT professionals with ethical and human values". To accomplish our mission, the department encourages students to apply their acquired knowledge and skills towards professional achievements in their career. The department also moulds the students to be socially responsible and ethically sound.

Introduction to the Programme

Machines are gaining more intelligence to perform human like tasks. Artificial Intelligence has spanned across the world irrespective of domains. MSc (Artificial Intelligence and Machine Learning) will enable to capitalize this wide spectrum of opportunities to the candidates who aspire to master the skill sets with a research bent. The curriculum supports the students to obtain adequate knowledge in the theory of artificial intelligence with hands-on experience in relevant domains with tools and techniques to address the latest demands from the industry. Also, c andidates gain exposure to research models and industry standard application development in specialized domains through guest lectures, seminars, industry offered electives, projects, internships, etc.

Programme Objective

● To acquire in-depth understanding of the theoretical concepts in Artificial Intelligence and Machine Learning
● To gain practical experience in programming tools for Data Engineering, Knowledge Representation, Artificial intelligence, Machine learning, Natural Language Processing and Computer Vision.
● To strengthen the research and development of intelligent applications skills through specialization based real time projects.
● To imbibe quality research and develop solutions to the social issues.

Programme Outcomes:
PO1 : Conduct investigation and develop innovative solutions for real world problems in industry and research establishments related to Artificial Intelligence and Machine Learning
PO2 : Apply programming principles and practices for developing automation solutions to meet future business and society needs.

PO3 : Ability to use or develop the right tools to develop high end intelligent systems
PO4 : Adopt professional and ethical practices in Artificial Intelligence application development
PO5 : Understand the importance and the judicious use of technology for the sustainability of the environment.

MAI417-3– Deep Learning

Total Teaching Hours for Trimester: 75
Max Marks: 100
Credits: 4

Course Description

The course introduces deep learning techniques, focusing on feedforward networks, CNNs, RNNs, and autoencoders. Students will learn regularization methods, optimization for long-term dependencies, and advanced model implementations. Lab exercises provide hands-on experience with Keras and TensorFlow. By the end, students will apply these techniques to real-world problems in computer vision and sequence processing.

Course Objectives

The main objective of this course is to make students comfortable with the tools and techniques required to handle large datasets. Several libraries and datasets publicly available will be used to illustrate the application of these algorithms. This will help students develop the skills required to gain experience in doing independent research and study.

Course Outcomes

Upon successful completion of the course, the student will be able to

CO1: Implement deep feedforward and backpropagation networks for classification.

CO2: Analyze regularization techniques for optimizing deep learning models.

CO3: Apply convolutional and recurrent neural networks for vision and sequence processing tasks.

CO4: Evaluate autoencoders for feature extraction and data compression.

CO5: Compare different deep learning architectures for performance improvement.

Unit-1 **Teaching Hours: 15**

DEEP FEEDFORWARD NETWORKS

An overview of ANN, Back Propagation Neural Networks, Deep Feedforward Networks: Deep network for Universal Boolean function representation, Classification and Approximation, Perceptron Learning.

Lab Exercises:

1. Demonstrate MLP in Keras/TensorFlow
2. Demonstrate Deep Feedforward Network

Unit-2 **Teaching Hours: 15**

REGULARIZATION FOR DEEP MODELS

Regularization for Deep models: L2 and L1 Regularization, Constrained Optimization and Under- Constrained Early Stopping, Parameter Tying and Parameter Sharing, Sparse representations, Dropout

Lab Exercises:

3. Demonstrate Regularization L1 and L2 for Deep learning model

Unit-3 **Teaching Hours: 15**

CONVOLUTIONAL NEURAL NETWORK

Introduction to convolution neural networks: stacking, striding and pooling, Applications in Computer Vision

Lab Exercises:

4. Demonstrate Convolution Neural Network

Unit-4 **Teaching Hours: 15**

RECURRENT NEURAL NETWORKS

Sequence Processing, Unfolding Computational Graphs, Training recurrent networks

The Long Short-Term Memory (LSTM), Optimization for Long- Term Dependencies, Encoder-Decoder Sequence-to-Sequence processing

Lab Exercises:

5. Demonstrate Short-Term Long Memory (LSTM)

Unit-5 **Teaching Hours: 15**

AUTOENCODERS

The architecture of autoencoders - the relationship between the Encoder, Bottleneck, and Decoder, how to train autoencoders? Types of autoencoders: Undercomplete autoencoders, Sparse autoencoders, Contractive autoencoders, Denoising autoencoders, Variational Autoencoders

Lab Exercises:

6. Demonstrate Sparse Autoencoders
7. Demonstrate Contractive Autoencoders

Text Books and Reference Books

[1] Deep Learning by Ian Goodfellow, Yoshua Bengio, Aaron Courville. MIT Press 2016.

[2] Simon J.D. Prince, Understanding Deep Learning: The Simple Math of Deep Learning, MIT Press 2024.

Essential Reading / Recommended Reading

[1] Deep Learning with Python by Francois Chollet. 2nd Edition, Manning Publications Co., 2020

[2] Introduction to Deep Learning by Eugene Charniak. The MIT Press 2019

[3] Dive into Deep Learning by Aston Zhang, Zack C. Lipton, Mu Li, Alex J. Smola. 2019

[4] AurélienGéron, "Hands-On Machine Learning with Scikit- Learn and TensorFlow", O'Reilly, 2022.

Web Resources:

[1] https://www.deeplearningbook.org/

[2] https://archive.ics.uci.edu/ml/datasets.php

CO – PO Mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7
CO1	3	3	2	2	2	1	2
CO2	2	2	1	1	1	1	2
CO3	3	3	2	1	2	1	2
CO4	2	2	1	1	1	1	2
CO5	3	3	2	2	2	2	3

LIST OF PROGRAMS

MSCAIML

Sl. no	Title of lab Experiment	Page number	RBT	CO
1	Learning the XOR Boolean Function Using an MLP		L3	CO1, 3
2			L3	CO2, 3
3			L3	CO3
4			L3	CO3
5			L3	CO3
6			L3	CO3, 4
7			L4	CO3
8			L3	CO3
9			L3	CO4
10			L5	CO4

Evaluation Rubrics:

Evaluation Rubrics for each program would be

Correctness and Demonstration and timely submission (5 =3+2marks)
Concept Clarity and modeling (3 marks)
Initiative & Effort ((2 marks)

Submission Guidelines:

- Make a copy of the lab manual template with your <name_reg:no_subject name >,
- Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as lab number.
- Keep updating your lab manual and show the lab manual of that particular lab for evaluation.

- Create a Git Repository in your profile <SPR lab-reg no> . Follow a different branch for each lab <Lab 1, Lab 2…>, and push the code to Git. The link should be provided in Google Classroom along with the PDF of the lab manual.
- Upload the PDF to Google Classroom before the deadline.

Lab 1

Lab Exercise I: Learning the XOR Boolean Function Using an MLP

Aim

1. To understand how to implement neural networks using different deep learning libraries (**Keras, PyTorch, and TensorFlow**).
2. To solve the non-linear XOR problem using an MLP and study the effect of hyperparameters such as learning rate, activation functions, number of neurons, and epochs on model performance.

Question

Implement an MLP to learn the XOR Boolean function

The XOR function takes two binary inputs (0 or 1) and produces a binary output (0 or 1) based on the following rule:

Input 1	Input 2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0

Steps:

1. **Create the Dataset**
   - Define input (X) and output (y) arrays for all 4 XOR combinations.
2. **Build an MLP**
   - Input layer: size 2 (for the two inputs).
   - Hidden layer: at least 2 neurons with **ReLU** or **Tanh** activation.
   - Output layer: 1 neuron with **sigmoid** activation for binary classification.
3. **Compile the Model / Define Loss and Optimizer**

- Use **Binary Cross-Entropy** loss.
- Use an optimizer of your choice (e.g., **Adam** or **SGD**).

4. **Train the Model**
   - Train the model on the XOR dataset.
   - Experiment with **epochs, learning rate, and number of neurons** to improve performance.
5. **Evaluate the Model**
   - Predict outputs for all 4 input combinations.
   - Check if the network correctly learns the XOR function.
6. **Implement Using Three Libraries**
   - Repeat the above steps using:
     1. **Keras (TensorFlow high-level API)**
     2. **PyTorch**
     3. **TensorFlow low-level API**

Additional Exercises (Optional):

- Plot the decision boundary for each implementation.
- Compare training curves and final accuracy between libraries.
- Discuss how changes in **learning rate, activation function, hidden layers, or epochs** affect learning.

Evaluation Rubrics

1. **Implementation** – 5 marks
2. **Complexity and Validation** – 3 marks
3. **Documentation & Writing the inference** – 2 marks

Submission Guidelines

1. Make a copy of the lab manual template with your <name_reg:no_subject name>
2. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
3. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
4. Create a **Git repository** in your profile <SPR lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
5. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
6. Upload the PDF to Google Classroom before the deadline.

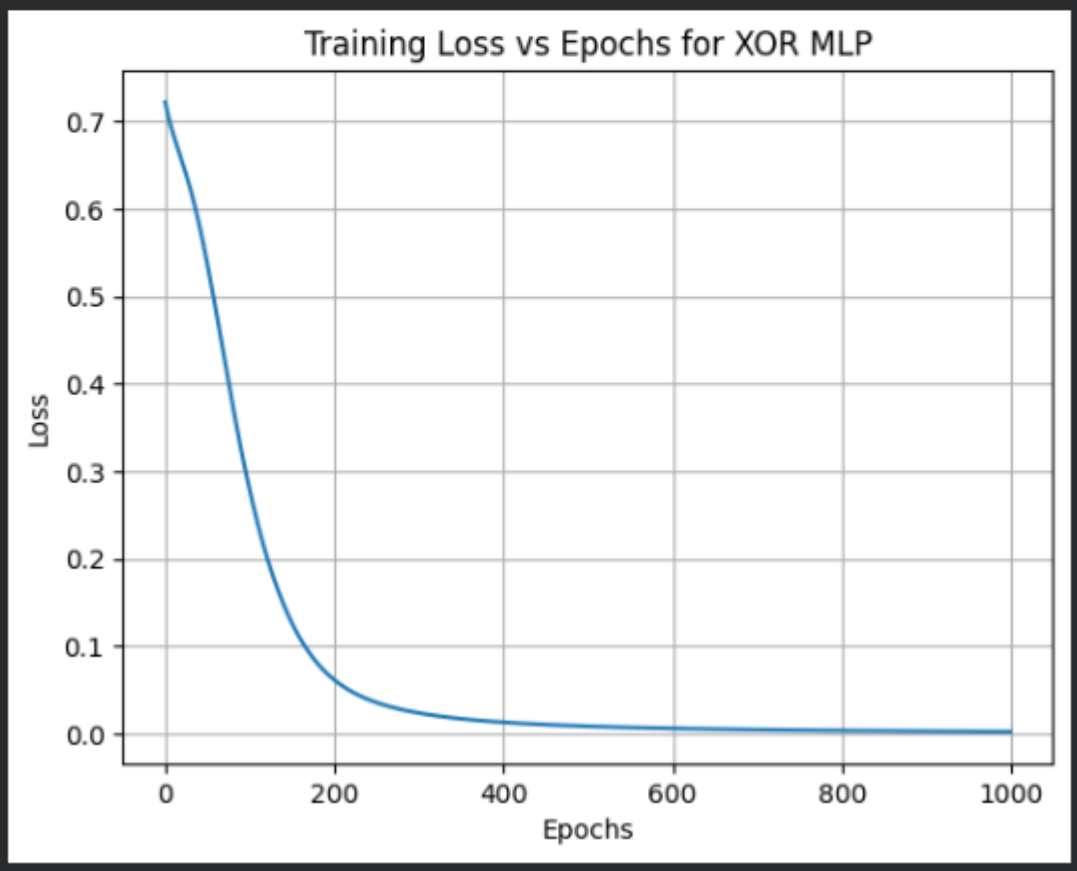
Code with Results

Keras

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# XOR data
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

# Model
model = Sequential([
    Dense(4, input_dim=2, activation='tanh'),
    Dense(1, activation='sigmoid')
])

# Compile
model.compile(
    optimizer=Adam(learning_rate=0.01),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(X, y, epochs=1000, verbose=0)

# Test
predictions = model.predict(X)
print("Predictions:\n", predictions.round())
```

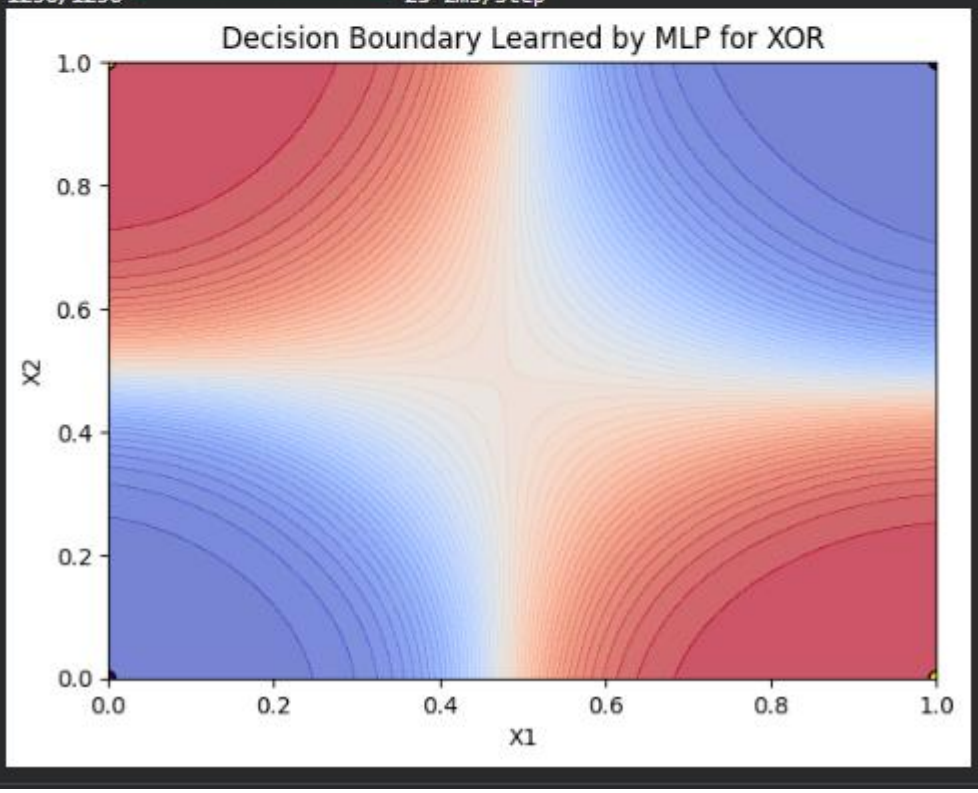
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/den
  super().__init__(activity_regularizer=activity_regularizer, **k
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 87ms/step
Predictions:
 [[0.]
 [1.]
 [1.]
```

```python
import matplotlib.pyplot as plt

plt.figure()
plt.plot(history.history['loss'])
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training Loss vs Epochs for XOR MLP")
plt.grid(True)
plt.show()
```



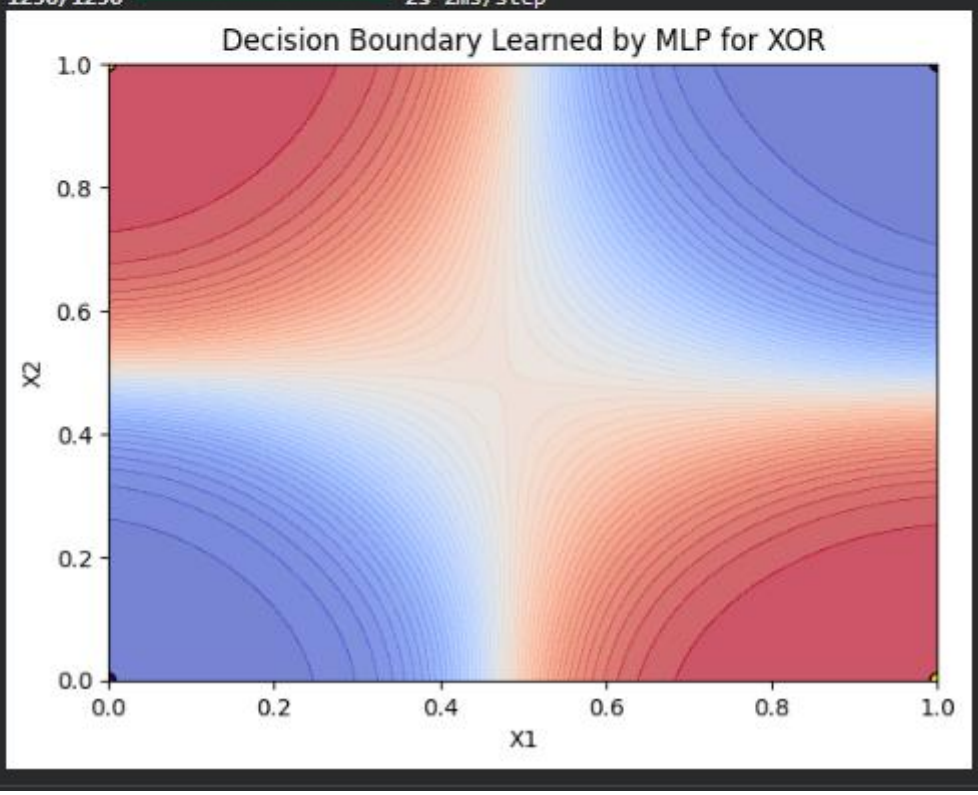
```python
# Create a grid of points
xx, yy = np.meshgrid(np.linspace(0, 1, 200),
                     np.linspace(0, 1, 200))

grid = np.c_[xx.ravel(), yy.ravel()]

# Predict on grid
Z = model.predict(grid)
Z = Z.reshape(xx.shape)

# Plot decision boundary
plt.figure()
plt.contourf(xx, yy, Z, levels=50, cmap='coolwarm', alpha=0.7)
plt.scatter(X[:,0], X[:,1], c=y.flatten(), edgecolors='k')
plt.xlabel("X1")
plt.ylabel("X2")
plt.title("Decision Boundary Learned by MLP for XOR")
plt.show()
```

```
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step
```



```
# Create a grid of points
xx, yy = np.meshgrid(np.linspace(0, 1, 200),
                     np.linspace(0, 1, 200))

grid = np.c_[xx.ravel(), yy.ravel()]

# Predict on grid
Z = model.predict(grid)
Z = Z.reshape(xx.shape)

# Plot decision boundary
plt.figure()
plt.contourf(xx, yy, Z, levels=50, cmap='coolwarm', alpha=0.7)
plt.scatter(X[:,0], X[:,1], c=y.flatten(), edgecolors='k')
plt.xlabel("X1")
plt.ylabel("X2")
plt.title("Decision Boundary Learned by MLP for XOR")
plt.show()
```

```
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step
```



```
activations = ['relu', 'tanh']
neurons = [2, 4, 8]

for act in activations:
    for n in neurons:
        model = Sequential([
            Dense(n, input_dim=2, activation=act),
            Dense(1, activation='sigmoid')
        ])
        model.compile(optimizer='adam', loss='binary_crossentropy
        model.fit(X, y, epochs=500, verbose=0)
        print(f"Activation={act}, Neurons={n}")
        print(model.predict(X).round())
```

```
Activation=relu, Neurons=2
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 72ms/step
[[0.]
 [1.]
 [1.]
 [0.]]
Activation=relu, Neurons=4
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
[[0.]
 [0.]
 [0.]
 [1.]]
Activation=relu, Neurons=8
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
[[0.]
 [1.]
 [1.]
 [0.]]
WARNING:tensorflow:5 out of the last 1254 calls to <function Tens
Activation=tanh, Neurons=2
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step
[[1.]
 [1.]
 [0.]
 [0.]]
WARNING:tensorflow:6 out of the last 1255 calls to <function Tens
Activation=tanh, Neurons=4
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step
[[0.]
 [1.]
 [1.]
 [1.]]
Activation=tanh, Neurons=8
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step
[[0.]
 [1.]
 [1.]
 [0.]]
```

PYTORCH

Pytorch

```python
import torch
import torch.nn as nn
import torch.optim as optim

# Data
X = torch.tensor([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
y = torch.tensor([[0.],[1.],[1.],[0.]])

# Model
class XORNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(2, 4)
        self.output = nn.Linear(4, 1)

    def forward(self, x):
        x = torch.tanh(self.hidden(x))
        x = torch.sigmoid(self.output(x))
        return x

model = XORNet()
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# Training
for epoch in range(1000):
    optimizer.zero_grad()
    output = model(X)
    loss = criterion(output, y)
    loss.backward()
    optimizer.step()

print("Predictions:\n", model(X).detach().round())
```

```
Predictions:
 tensor([[0.],
        [1.],
        [1.],
        [0.]])
```

tensorflow

```python
import tensorflow as tf
import numpy as np

# XOR dataset
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([[0],[1],[1],[0]], dtype=float)

# MLP model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(2, activation='tanh', input_shape=(2,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compile model
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train model
model.fit(X, y, epochs=1000, verbose=0)

# Predictions
print("Predicted Output:")
print(model.predict(X).round())
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Predicted Output:
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step
[[0.]
 [1.]
 [1.]
```

Conclusion /inference

From the implementation and experiments performed using a Multi-Layer Perceptron (MLP) in TensorFlow, it is observed that the XOR Boolean function cannot be solved using a linear model and requires at least one hidden layer with a non-linear activation function. The model successfully learned the XOR mapping when an appropriate activation function (tanh or ReLU), sufficient number of hidden neurons (minimum two), suitable learning rate, and adequate number of training epochs were used. The experimental results also show that the choice of hyperparameters significantly affects convergence speed and prediction accuracy. Hence, this experiment demonstrates the importance of hidden layers, non-linearity, and proper hyperparameter tuning in solving non-linearly separable problems using neural networks.